



Video Presentation

Exploring classification and visualization of
emissions data in Malaysia

Prepared by:

Muhammad Syahmi Faris bin Rusli A23CS0138

Ahmad Adib Zikri bin A.Mazlam A23CS0205

Muhammad Mukhritz Al Iman bin Mohd Raffi A23CS0250





Project Objectives

Main Objective:

~ To analyze vehicle emissions in Malaysia by integrating car registration and fuel consumption data, and to apply machine learning techniques for emission classification."

◆ Sub-Objectives:

- Merge and clean real-world datasets
- Perform exploratory data and summary analysis
- Classify states based on emission level using ML models
- Visualize findings to support environmental insight



Why Analyze Carbon Emissions?

Motivation:

- Climate change and pollution are major global issues.
- Transportation is one of the largest contributors to CO₂ emissions.
- Understanding which states emit more can guide:
 - Sustainable transport planning
 - Environmental policymaking
 - Public awareness

Real-World Application:

- Governments and researchers use such insights to regulate vehicle imports, subsidize EVs, or adjust fuel taxes.

Data Integration and Cleaning

We analyze cars_2025.csv and fuelconsumption.csv for insights

Step Description

- 1) Load datasets (cars_2025.csv and FuelConsumption.csv)
- 2) Normalize columns (maker, model) to ensure merge compatibility
- 3) Merge datasets using left join
- 4) Handle missing CO₂ emissions (identify unmatched rows and drop them)

```
import pandas as pd
```

1. Loading two datasets into panda Dataframes

```
# Phase I: Data Preparation & Cleaning
```

```
# 1. Load datasets
```

```
cars = pd.read_csv('cars_2025.csv')  
fuel = pd.read_csv('FuelConsumption.csv')
```

2. Creating new columns with cleaned and standardized versions of maker and model names

```
# 2. Normalize join keys
```

```
cars['maker_norm'] = cars['maker'].str.strip().str.lower()  
cars['model_norm'] = cars['model'].str.strip().str.lower()  
fuel['maker_norm'] = fuel['MAKE'].str.strip().str.lower()  
fuel['model_norm'] = fuel['MODEL'].str.strip().str.lower()
```

3. Join cars with fuel using the normalized maker and model fields.

```
# 3. Merge registrations with emissions data
```

```
merged = cars.merge(  
    fuel,  
    how='left',  
    on=['maker_norm', 'model_norm']  
)
```

4. Filter out rows where no CO₂ emissions data was found

Create a new DataFrame that only contains rows with valid CO₂ data

```
# 4. Handle missing CO2 values
```

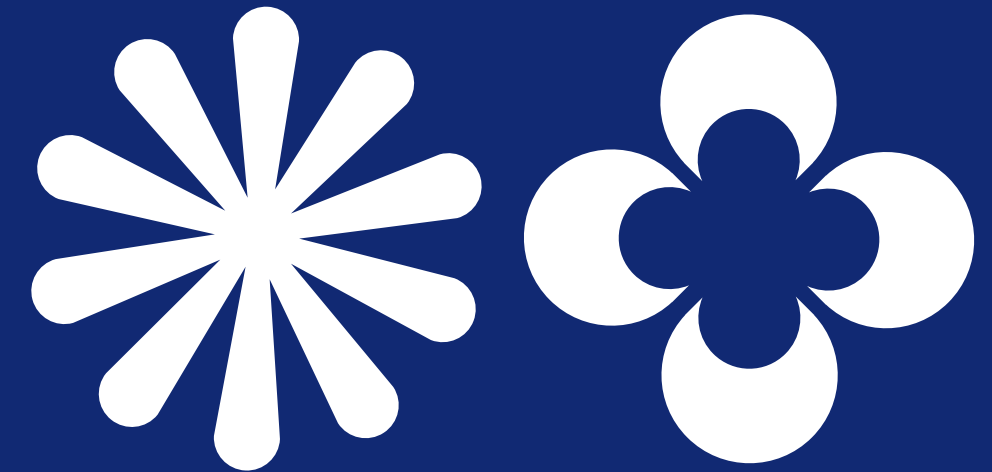
```
# a) isolate unmatched for manual review
```

```
unmatched = merged[ merged['CO2EMISSIONS'].isna() ]
```

```
# b) drop rows with missing CO2 for clean analysis
```

```
merged_clean = merged.dropna(subset=['CO2EMISSIONS']).copy()
```


Cleaning Validation & Emission Averages



5. Remove any completely identical rows in the clean dataset

```
# 5. Optional: check for and drop exact duplicate rows
merged_clean.drop_duplicates(inplace=True)
```

6. Calculating total registration, matched and didn't match with emissions data

```
# 6. Aggregations & summaries
```

```
# 6a. Counts summary
```

```
total_regs = len(cars)
matched_regs = len(merged_clean)
unmatched_regs = len(unmatched)
```

```
print("Total registrations: ", total_regs)
print("Matched records: ", matched_regs)
print("Unmatched records: ", unmatched_regs)
```

```
# 6b. Average CO2 emissions by fuel type
```

```
avg_by_fuel = (
    merged_clean
    .groupby('fuel')['CO2EMISSIONS']
    .mean()
    .sort_values()
    .reset_index()
)
print("\nAverage CO2 by fuel type:")
print(avg_by_fuel.to_string(index=False))
```

```
# 6c. Average CO2 emissions by state
```

```
avg_by_state = (
    merged_clean
    .groupby('state')['CO2EMISSIONS']
    .mean()
    .sort_values()
    .reset_index()
)
print("\nAverage CO2 by state:")
print(avg_by_state.to_string(index=False))
```

```
Total registrations: 263578
Matched records: 9212
Unmatched records: 251480
```

Average CO₂ by fuel type:

fuel	CO2EMISSIONS
petrol	196.747879
hybrid_petrol	200.969723
diesel	221.150000

Average CO₂ by state:

state	CO2EMISSIONS
Pahang	182.205882
Sabah	182.220930
Perlis	182.387755
Kelantan	183.099099
Kedah	185.548571
Terengganu	186.218182
Melaka	191.977011
Negeri Sembilan	192.200000
Sarawak	193.130719
Rakan Niaga	195.616549
Pulau Pinang	197.304659
Selangor	197.992701
Johor	199.213542
Perak	199.695312
W.P. Kuala Lumpur	208.012796

StepDescription

5) Drop exact duplicates to avoid double-counting

6a) Print total registrations, matched & unmatched rows

6b) Calculate average CO₂ by fuel type

6c) Calculate average CO₂ by state

Hierarchical Summary by State and Fuel Type

Step Description

7a) Group CO₂ by (state, fuel) using MultiIndex

7b) Show example (Pahang CO₂ by fuel type)

7c) Pivot table of (state × fuel) with CO₂ averages

7. Calculate the average emissions for every combination of state and fuel type, printing the average CO₂ by fuel type only for Pahang AND turn the hierarchical index into a pivot table

```
# 7. Hierarchical indexing & pivot

# 7a. Create MultiIndex of (state, fuel)
mi = (
    merged_clean
    .groupby(['state', 'fuel'])['CO2EMISSIONS']
    .mean()
)

# 7b. Example partial indexing for one state
print("\nPahang CO2 by fuel type:")
print(mi.loc['Pahang'])

# 7c. Pivot to wide form: states as rows, fuel types as columns
emissions_table = mi.unstack()
print("\nPivot table (state × fuel) of average CO2:")
print(emissions_table.head().to_string())
```

```
Pahang CO2 by fuel type:
fuel
hybrid_petrol    188.166667
petrol           181.411111
Name: CO2EMISSIONS, dtype: float64
```

```
Pivot table (state × fuel) of average CO2:
fuel          diesel  hybrid_petrol    petrol
state
Johor          NaN      217.833333    197.520833
Kedah          NaN      177.750000    185.922156
Kelantan       NaN      184.000000    183.056604
Melaka         NaN      223.000000    189.262500
Negeri Sembilan  NaN      198.583333    190.984127
```